

Session 6 Speaker Script — COMPLETE WITH FULL TALKING POINTS

Fine-Tuning & Specialization

Presenter: Dinesh Wadhvani | **Session:** 6 of 12 | **Duration:** ~60 minutes

Purpose: Teach LLMs your domain through targeted training

Slide	Title	Time
1	Title Slide	~1 min
2	Today's Journey	~2 min
3	What is Fine-Tuning?	~5 min
4	Fine-Tuning vs RAG	~5 min
5	When to Fine-Tune	~8 min
6	How Fine-Tuning Works	~8 min
7	Data Preparation	~8 min
8	Training Process	~5 min
9	Evaluation & Metrics	~8 min
10	Real-World Examples	~8 min
11	RAG + Fine-Tuning	~5 min
12	Lab Overview	~3 min
13	Key Takeaways	~5 min
14	Session 7 Preview	~2 min
—	Lab Exercise	~12 min

01. TITLE SLIDE

~1 minute

WHAT'S ON SCREEN

Session 6 label (blue) • Title: 'Fine-Tuning & Specialization' • Subtitle: 'Teaching LLMs Your Domain' • Session 6 of 12

SPEAKING SCRIPT

Welcome to Session 6. We're at a critical junction in the course. Sessions 1-5 taught you how to use LLMs: prompt them, structure their output, retrieve context to ground them. Now we're going deeper: how to change the LLM itself.

RAG grounds models in documents. Fine-tuning teaches them your domain. They're complementary. Together, they create AI that's both knowledgeable AND trustworthy.

Here's the key insight: base LLMs like GPT-4 are generalists. They know a bit about everything. But your domain is specific. Hiring has specific terminology, specific patterns. Legal has precedent and exact language. Medicine has evidence-based practices. Customer service has your company's tone and values.

Fine-tuning is how you make the model specialize. Not by changing the model's weights randomly. But by training it on examples from your domain so it learns the patterns that matter to you.

By end of today, you'll understand how fine-tuning works, when to use it, how to evaluate if it's working, and you'll build a simple fine-tuned model for your domain. In the lab, you'll create training data, understand the formats, and simulate what a real fine-tuning process looks like.

Let's start.

DELIVERY TIPS

This is the transition to 'building your own model.' High energy—this is exciting. • Emphasize RAG + fine-tuning together are powerful. • 'Your domain is specific—that's why fine-tuning matters.'

02. TODAY'S JOURNEY

~2 minutes

WHAT'S ON SCREEN

14-item agenda • Items: What is FT, FT vs RAG, When to FT, How it works, Data prep, Training, Evaluation, Real examples, Combined with RAG, Lab

SPEAKING SCRIPT

Quick roadmap. We start with five minutes on what fine-tuning is. Simple concept: showing the model lots of examples and asking it to learn from them. But the implications are profound.

Then five minutes on fine-tuning vs RAG. Which do you use? When? They're different tools for different problems.

Eight minutes on when to fine-tune. Not every problem needs it. Sometimes it's overkill. Sometimes it's essential. We'll cover decision framework.

Eight minutes on how fine-tuning works mechanically. What's actually happening inside the model? Why does it work?

Eight minutes on data preparation. This is where most of the work is. Getting data in the right format, the right quality, the right amount.

Five minutes on the training process. How do you actually fine-tune? What tools? What does the process look like?

Eight minutes on evaluation and metrics. How do you know if your fine-tuned model is better than the base model? What do you measure?

Eight minutes on real-world examples. Companies using fine-tuning in production. What problems are they solving?

Five minutes on combining RAG with fine-tuning. This is where the magic happens. Grounded AND specialized.

Then lab—12 minutes. You'll create training data for your domain, understand the formats, and build a simple fine-tuned model.

Then takeaways and Session 7 preview. If time gets tight, we compress the mechanics section. We protect data prep, evaluation, and lab.

DELIVERY TIPS

Move briskly—lots of ground to cover. • Emphasize that fine-tuning + RAG together are the winning combination. • Preview the lab: you'll learn what real companies do.

03. WHAT IS FINE-TUNING?

~5 minutes

WHAT'S ON SCREEN

Definition box • 'Training a pre-trained model on domain-specific examples' • Comparison: Pre-training vs Fine-tuning • Impact on model behavior

SPEAKING SCRIPT

Fine-tuning is training a model that's already been trained, using new data from your specific domain.

ANALOGY:

Pre-training (Session 2 concept): Teaching someone everything about the world. Billions of facts, patterns, language. This is what happens when LLMs are initially trained on internet data.

Fine-tuning: Taking that educated person and saying, 'Now, let's focus. For the next month, study only legal cases.' Or 'Study only medical journals.' Or 'Study only our company's customer interactions.'

Result: The person becomes a specialist. Still has general knowledge, but now deeply specialized in one domain.

TECHNICAL DEFINITION:

Fine-tuning is taking a pre-trained model and continuing its training on new, domain-specific data. The model's weights are updated. It learns new patterns specific to your domain.

WHAT CHANGES:

The model learns: How your domain talks. What patterns are important. What's right and wrong in your context.

What doesn't change: The base architecture (transformer from Session 2) stays the same. The context window stays the same. The reasoning ability stays the same.

REAL EXAMPLE: CUSTOMER SERVICE

Base GPT-4: Can write helpful responses to generic customer complaints.

Fine-tuned on your company's data: Writes responses in your company's voice, following your policies, using your terminology, with your empathy level.

Same model. Different training data. Different behavior.

HOW IT WORKS (SIMPLE VERSION):

You give the model: Question-Answer pairs from your domain. Hundreds or thousands.

Model trains: 'When the input is like this, the output should be like that. I see a pattern.'

Result: Model gets better at your specific task.

FINE-TUNING VS PROMPTING:

Prompting (Session 3): Tell the model in the prompt what you want. 'Answer like a lawyer.' 'Use this tone.' Instructions in the prompt.

Fine-tuning: The model learns from examples what you want. No need to tell it every time. It just knows.

Example difference:

Prompting: Prompt says 'Be professional. Use lawyer terminology.' Every prompt includes this.

Fine-tuning: Model trained on legal examples. Just ask the question. Model automatically answers like a lawyer.

WHEN TO USE EACH:

Prompting: Quick, cheap, flexible. Use for one-off requests or when requirements change often.

Fine-tuning: Slower, more expensive, permanent. Use when you have consistent domain and lots of examples.

DELIVERY TIPS

The analogy (educating a person, then specializing) is powerful. Stick with it. • Show the difference: Prompting = instructions per query, Fine-tuning = learned from examples. • Real example (customer service) makes it concrete. • Students should understand: fine-tuning is an investment, not a quick fix.

04. FINE-TUNING vs RAG

~5 minutes

WHAT'S ON SCREEN

Comparison table • RAG: Retrieves documents, answers based on context • Fine-tuning: Learns from examples, bakes knowledge into weights • When to use each

SPEAKING SCRIPT

This is crucial. Fine-tuning and RAG are different. Sometimes you use one, sometimes the other, sometimes both. Understanding when to use which is key.

RAG (FROM SESSION 5):

What it does: When a question comes in, retrieve relevant documents, give to model as context.

Mechanism: Knowledge is external (in documents). Retrieved at query time.

Speed: ~1 second (need to retrieve first)

Cost: Cheap (only retrieval cost, no retraining)

Flexibility: Can change knowledge base anytime

Freshness: Can always update documents

FINE-TUNING (THIS SESSION):

What it does: Train model on domain examples so it learns patterns.

Mechanism: Knowledge is internal (in model weights). Learned during training.

Speed: Fast (no retrieval needed)

Cost: Expensive (training is costly)

Flexibility: Once trained, can't change easily. Need to retrain.

Freshness: Only as fresh as training data. Need retraining for updates.

DECISION FRAMEWORK:

Use RAG when:

- Knowledge is frequently updated (today's news, current prices, new documents)■r/>- You have lots of documents (millions)■r/>- You need to cite sources■r/>- You want to keep knowledge external and auditable■r/>Example: Bing Chat (retrieves fresh web results)■r/>

Use Fine-tuning when:

- Patterns are stable (legal language, medical terminology, company tone)■r/>- You have lots of training examples (100s or 1000s)■r/>- Speed matters (no retrieval overhead)■r/>- You want to bake knowledge into the model

Example: Legal AI trained on years of case law

Use BOTH when:

- Domain is specialized (legal, medical) → fine-tune for domain
- Plus knowledge is large and changing → RAG for current cases
- Result: Specialized model that cites current sources

Example: Medical AI that's fine-tuned on medical language PLUS retrieves latest research

REAL EXAMPLE: LEGAL TECH COMPANY

Approach 1: Pure RAG

- Store all case law in vector database
- Retrieve relevant cases when queried
- LLM synthesizes them■r/>- Pro: Always current, fully sourced
- Con: Slow (need to retrieve), cost (retrieval overhead)■r/>

Approach 2: Pure Fine-tuning

- Fine-tune model on years of case law
- When queried, model directly generates analysis
- Pro: Fast, no retrieval needed
- Con: Not fresh, can't cite specific cases unless in training data, expensive to retrain

Approach 3: Both (Best)

- Fine-tune model on legal language and reasoning patterns
- At query time, also retrieve most relevant cases as context
- Model generates analysis informed by both learned patterns AND current cases
- Pro: Fast, current, specialized, can cite sources
- Con: Complex, requires both fine-tuning and RAG infrastructure

WHICH COMPANIES USE WHICH:

Fine-tuning heavy: Internal specialized tools, stable domains, real-time constraints

RAG heavy: News, current events, large knowledge bases

Both: Serious production systems (medical, legal, finance)

FOR YOUR HIRING DOMAIN:

Could use RAG alone: Retrieve similar resumes, match to job reqs

Could use fine-tuning alone: Train on hiring examples, learn to evaluate candidates

Best approach: Fine-tune on hiring patterns, RAG on current resumes + job descriptions

DELIVERY TIPS

Decision framework is practical—students will use this. • Legal tech example showing all three approaches is powerful. • Emphasize: they're tools for different problems, not competitors. • Connection back: 'RAG grounds in documents. Fine-tuning teaches the model. Together they're powerful.'

05. WHEN TO FINE-TUNE

~8 minutes

WHAT'S ON SCREEN

Decision tree • Four key criteria • Cost-benefit analysis • Real-world decision examples

SPEAKING SCRIPT

Before you fine-tune, ask yourself these questions. Because fine-tuning is not cheap or fast. You need good reasons.

CRITERION 1: DO YOU HAVE ENOUGH DATA?

Minimum: 100 examples

Good: 500-1000 examples

Excellent: 10,000+ examples

Why? Models need lots of examples to learn patterns. Too few = overfitting (model memorizes rather than learns).

If you don't have enough data: Don't fine-tune. Use prompting or RAG instead.

CRITERION 2: IS THE DOMAIN DISTINCT FROM GENERAL KNOWLEDGE?

Distinct: Legal language, medical terminology, company-specific processes

Not distinct: General customer service, writing emails, common questions

Why? If the domain is general, the base model probably already handles it well. Fine-tuning won't improve much.

Test: Does prompting already work pretty well? If yes, fine-tuning helps but might not be worth the cost.

CRITERION 3: IS SPEED CRITICAL?

Critical: Real-time systems, chatbots that need millisecond responses

Not critical: Batch processing, analysis that can take seconds

Why? Fine-tuning removes retrieval overhead. If speed matters, it's worth it. If not, RAG works fine.

CRITERION 4: IS THE KNOWLEDGE STABLE?

Stable: Legal precedent (changes slowly), medical terminology (stable), company policies (relatively stable)

Unstable: Current events, stock prices, news, constantly changing regulations

Why? Fine-tuning bakes knowledge into the model. If knowledge changes frequently, you need to retrain frequently. Expensive. RAG is better for unstable knowledge.

DECISION TREE:

Q1: Do you have 100+ examples?

No → Don't fine-tune. Use prompting/RAG.

Yes → Continue

Q2: Is domain distinct?

No → Prompting might be enough. Skip fine-tuning.

Yes → Continue

Q3: Is speed critical?

No → RAG probably sufficient. Consider fine-tuning only for domain specialization.

Yes → Fine-tuning attractive. Continue.

Q4: Is knowledge stable?

No → RAG better. Don't fine-tune.

Yes → FINE-TUNE. Worth the cost and complexity.

COST-BENEFIT ANALYSIS:

COSTS of fine-tuning:

- Time: Weeks to collect and label data
- Money: Training costs (can be \$100-\$10,000+ depending on model and data size)
- Complexity: Need to manage fine-tuned model, evaluate it, potentially retrain it
- Inflexibility: Once trained, hard to change quickly

BENEFITS of fine-tuning:

- Performance: Often 5-20% improvement over base model on domain-specific tasks
- Speed: No retrieval latency (faster than RAG) ■r/>- Specialization: Model truly understands your domain
- Cost savings: After initial investment, no per-query cost for retrieval

BREAKEVEN ANALYSIS:

If you're going to run your model: ■r/>- 1,000 times: RAG probably cheaper ■r/>- 100,000 times: Fine-tuning might break even ■r/>- 1 million+ times: Fine-tuning definitely cheaper (no per-query retrieval cost)

REAL-WORLD EXAMPLES:

EXAMPLE 1: STARTUP WITH 50 CUSTOMER SERVICE TICKETS

Decision: Don't fine-tune

Why: Not enough data (need 100+), and you don't have enough query volume to justify cost

Do instead: Use prompting with good templates, maybe add RAG if you have FAQ docs

EXAMPLE 2: LEGAL FIRM WITH 5000 CASE ANALYSES FROM PAST 10 YEARS

Decision: Fine-tune

Why: Enough data (5000 examples), domain is distinct (legal language), knowledge is stable (precedent changes slowly), speed helps (lawyers need quick answers)

Cost-benefit: Investment of \$2,000-5,000 in fine-tuning. Used 100,000+ times per year. Pays for itself in months.

EXAMPLE 3: NEWS ORGANIZATION TRACKING TODAY'S EVENTS

Decision: Don't fine-tune (use RAG instead)

Why: Knowledge is unstable (constantly changing), would need to retrain daily ■r/>Do instead: RAG on today's news articles, constantly updated

EXAMPLE 4: BANK WITH 100,000 CUSTOMER INTERACTIONS

Decision: BOTH—fine-tune AND RAG

Why: Enough data for fine-tuning (100k examples). Domain is stable (banking terminology). Plus use RAG for current regulations, account info, recent transactions. ■r/>Result: Fast specialized model (fine-tuned) with current context (RAG).

FOR YOUR DOMAIN (HIRING):

Do you have 100+ hiring-related examples? Probably yes (resumes, job descriptions, hiring feedback)

Is domain distinct? Yes (hiring terminology, evaluation criteria)

Is speed critical? Maybe (depends on your use case)

Is knowledge stable? Mostly yes (evaluation criteria), some unstable (current job market, new roles)

Verdict: Good candidate for fine-tuning, especially combined with RAG for current job info.

DELIVERY TIPS

Decision tree is practical—students will reference this when building real systems. • Cost-benefit analysis is honest: fine-tuning is expensive. Help students think economically. • Real examples across different industries shows this isn't one-size-fits-all. • Prompt skepticism: 'Is fine-tuning really needed?' is a good question. • Hiring domain example ties back to their project throughout the course.

06. HOW FINE-TUNING WORKS

~8 minutes

WHAT'S ON SCREEN

Fine-tuning process visual • Pre-training vs Fine-tuning comparison • Weight updates, loss function, convergence • Learning rate and epoch concepts

SPEAKING SCRIPT

Let's talk about the mechanics. How does fine-tuning actually change the model?

QUICK REVIEW FROM SESSION 2:

Remember: LLMs have weights. Billions of them. These weights encode learned patterns. During pre-training, these weights are adjusted to predict the next token better.

Fine-tuning: Takes those already-trained weights and adjusts them further based on your data.

THE PROCESS:

Start: You have a pre-trained model (GPT-4, LLaMA, etc.)

Step 1: Prepare examples

Your training data is pairs: (input, expected output)

Example: (Resume text, Hiring decision: 'Strong match')

Step 2: Forward pass

Model reads input, generates output prediction

Example: Model predicts 'Weak match'

Step 3: Calculate loss

Loss = How wrong was the prediction?

Expected: 'Strong match', Predicted: 'Weak match' → Loss is high

Step 4: Backward pass

Calculate gradients: Which weights should change to reduce loss?

Step 5: Update weights

Adjust weights slightly in direction that reduces loss

Learning rate controls how much adjustment (too much = overshooting, too little = slow learning)

Step 6: Repeat

Do this for all training examples. Multiple times (called epochs).

Result: Weights are adjusted toward your domain.

WHAT'S CHANGING:

Not the whole model. Only the final layers get significant updates. Early layers (basic language understanding) stay mostly same. Final layers (task-specific reasoning) get updated most.

Why? Because early layers already learned fundamental language patterns. You don't need to retrain that. You only need to adapt later layers to your domain.

LEARNING RATE AND EPOCHS:

Learning rate: How aggressively weights change

- Too high: Model overshoots, accuracy bounces around
- Too low: Learning is slow, takes forever
- Just right: Smooth improvement

Typical: 0.00001 to 0.001 (very small)

Epochs: How many times you go through the training data

- 1 epoch: One pass through all examples
- 3-10 epochs: Common for fine-tuning
- Too many: Overfitting (memorizes, doesn't generalize)

OVERFITTING DANGER:

If you train too long on too little data, model memorizes.

Example: If you have only 50 customer service examples and train for 100 epochs, model memorizes those 50 examples exactly. Doesn't generalize to new queries.

Prevention: Use validation set. Separate 20% of training data as validation. After each epoch, test on validation set. Stop when validation accuracy plateaus (sign of overfitting).

PRACTICAL EXAMPLE: LEGAL AI

Pre-trained GPT-4: Knows general language, can write about many topics.

Fine-tuning process:

1. Collect 1000 legal cases with analysis (input: case description, output: legal analysis)
2. Show model case 1, compare prediction to expert analysis, calculate loss
3. Adjust weights to predict more like expert
4. Repeat for cases 2-1000
5. Second epoch: Go through 1000 cases again, refine weights further
6. Third epoch: Same
7. After 3 epochs, check validation set. If accuracy is good, stop.

Result: Model's final layers now have updated weights optimized for legal analysis.

MEMORY AND COMPUTE REQUIREMENTS:

Fine-tuning requires: GPU, lots of memory, specialized infrastructure

Why? You're doing forward and backward passes on all training examples, multiple times. Requires computation.

Cost: Typically \$100-10,000 depending on model size and data size

Time: Can range from hours to days

PARAMETER-EFFICIENT FINE-TUNING:

Full fine-tuning: Update all weights (expensive)

Parameter-efficient alternatives: Update only a small percentage of weights (cheap)

- LoRA (Low-Rank Adaptation): Update ~1% of weights, 90% of performance, 10% of cost
- Prefix tuning: Add small trainable tokens, keep model weights frozen

These techniques exist because full fine-tuning is expensive. For most domains, 1% of weights is enough.

DELIVERY TIPS

Mechanics explanation should start simple, build up. • The five-step process (forward, calculate loss, backward, update, repeat) is the core. • Overfitting is a real risk—make this concrete. • Learning rate and epochs are practical—students need to know these. • Parameter-efficient methods show this is becoming more accessible. • Legal example ties to real-world application.

07. DATA PREPARATION

~8 minutes

WHAT'S ON SCREEN

Data requirements • Format specifications • Quality criteria • Cleaning process • Example training pairs

SPEAKING SCRIPT

Here's the truth: 80% of fine-tuning work is data preparation. Not the training. The data.

This is where most projects fail. People want to jump to training. But garbage data → garbage model.

DATA FORMAT:

Most fine-tuning APIs expect JSONL format (JSON Lines):

```
{'prompt': 'What are Python basics?', 'completion': 'Python is a programming language known for simplicity and readability.'}
{'prompt': 'Explain machine learning', 'completion': 'Machine learning is a subset of AI where systems learn from data patterns.'}
```

Each line is one training example.

FORMAT SPECIFICS:

Prompt: The input. What the model will receive.

Completion: The output. What model should generate.

If fine-tuning a chat model:

```
{'messages': [{'role': 'user', 'content': 'Classify this resume: [resume]'}, {'role': 'assistant', 'content': 'Classification: Strong match'}]}
```

DATA REQUIREMENTS:

Minimum: 100 examples (can work with less, but risky)

Good: 500-1000 examples

Excellent: 10,000+ examples

Quality: Each example must be correct. Wrong examples teach wrong patterns.

Variety: Examples should be diverse. If all examples are similar, model overfits to that pattern.

DATA COLLECTION:

Sources:

- Existing company data: Past customer interactions, decisions, documents
- Public datasets: Research datasets, benchmarks, open data
- Synthetic data: Generated examples (careful—can be biased)
- Manual creation: Hand-written examples (expensive but highest quality)

Most projects use combination.

DATA CLEANING:

Step 1: Remove duplicates

If same prompt-completion pair appears twice, remove duplicates

Step 2: Remove errors

Review data. Delete examples where completion is wrong or incomplete

Step 3: Normalize format

Ensure all prompts follow similar structure

Ensure all completions follow similar format

Step 4: Handle edge cases

Very long examples: Might exceed context window. Truncate or split.

Very short examples: Might not teach useful patterns. Consider removing.

Ambiguous examples: If you're not sure what right answer is, remove it.

DATA QUALITY METRICS:

Completeness: No missing fields?

Correctness: Are answers actually right?

Consistency: Similar inputs → similar outputs?

Diversity: Good spread of examples across domain?

REAL EXAMPLE: HIRING DOMAIN

Raw data: 500 past hiring decisions (resume + decision: hired or rejected)

Cleaning process:

1. Remove 50 duplicates → 450 examples
2. Remove 30 where decision unclear → 420 examples
3. Remove 20 where resume data corrupted → 400 examples
4. Normalize all resumes to same format → 400 examples
5. Split into: 320 training, 80 validation

Result: 320 clean training examples. Not huge, but workable.

Format:

```
{'prompt': 'Evaluate this resume for Software Engineer role. [resume text]', 'completion': 'Recommendation: STRONG MATCH. Reasoning: [reasoning]'}
```

DATA SPLIT:

Training set: 80% (learn from these)

Validation set: 20% (test if learning is working)

Why not 100% training? Because validation set tells you if model is overfitting. If training accuracy is 95% but validation is 60%, you're overfitting.

SYNTHETIC DATA WARNING:

You can generate synthetic training data using LLMs themselves. Tempting—cheap and fast.

Risk: If you generate examples with wrong patterns, model learns those patterns.

Example: 'Generate 100 hiring decisions.' If generator is biased, synthetic data will be biased too.

Rule: Mix synthetic with real. Don't use 100% synthetic.

DATA LABELING:

If you don't have ready-made examples, you need to label data.

Cost: Paying humans to label is expensive. ~\$1-5 per example.

Crowdsourcing: Services like Amazon Mechanical Turk, Upwork

Internal labeling: Your team labels (more consistent, slower)

For hiring domain:

- Collect 500 resumes
- Have hiring team review each and label: 'Strong', 'Weak', 'Borderline'■r/>- Pay: ~\$500-2000 in labeling■r/>- Result: 500 labeled training examples

FINAL CHECKLIST:

- Have 100+ examples? (Preferably 500+)
- Format correct? (JSONL or whatever your API requires)
- Duplicates removed?
- Errors removed?
- Normalized?
- Edge cases handled?
- Split into train/validation?
- Quality reviewed?

If yes to all: Ready to fine-tune.

DELIVERY TIPS

Data prep is unsexy but essential. Make students understand: good data > good training. • Real hiring example shows practical process. • Data quality metrics are important—establish expectation for quality. • Overfitting warning is critical. • The checklist is actionable—students can use it. • Be honest: this is where the work happens, not training.

08. THE TRAINING PROCESS

~5 minutes

WHAT'S ON SCREEN

Training flow • Tools and platforms shown • Timeline and resources • Monitoring training progress

SPEAKING SCRIPT

Once your data is ready, the training itself is relatively straightforward. Many tools exist to make it easy.

TOOLS FOR FINE-TUNING:

OpenAI Fine-tuning API:

- Pros: Simple, handled for you, good results
- Cons: Proprietary (locked into OpenAI)
- Cost: ~\$3 per 1M training tokens, \$0.03 per 1K completion tokens during inference
- Time: Hours to days

Anthropic Claude (being added):

- Simple API, good results, transparent pricing

Open-source (LLaMA fine-tuning):

- Pros: Full control, free
- Cons: Requires technical setup, GPU expensive, slower
- Tools: Hugging Face, LitGPT, or custom

No-code platforms:

- Replicate, Hugging Face, Modal
- Pros: Easy, no infrastructure management
- Cons: Less flexible, more expensive

TYPICAL WORKFLOW WITH OPENAI API:

Step 1: Prepare data (we covered this)

Step 2: Upload to OpenAI

```
openai api fine_tunes.create -t data/train.jsonl -v data/validation.jsonl --model gpt-3.5-turbo
```

Step 3: Monitor training

OpenAI shows progress: Accuracy improving? Loss decreasing? Overfitting?

Takes 1-4 hours typically

Step 4: Evaluate results

Test fine-tuned model on new data (not seen during training)

Compare to base model: Is fine-tuned better?

Step 5: Deploy

If better, use fine-tuned model. If not, investigate why.

MONITORING DURING TRAINING:

Loss: Should decrease over time

If loss not decreasing: Data quality issue or learning rate too low

Training vs Validation accuracy:

If training >> validation: Overfitting
If both low: Maybe not enough data or data quality issue

Time to convergence:

Usually 1-4 hours for reasonable dataset

If takes days: Something wrong

COST EXAMPLE:

Fine-tuning a model on 1000 examples (~500K tokens):

Training cost: ~\$1.50

Per-query inference: ~\$0.03

If using model 10,000 times: \$300 inference cost

Total: ~\$300, very reasonable

REAL-WORLD TIMELINE:

Day 1: Collect raw data (1-10 hours) Day 2: Clean and format (2-8 hours) Day 3: First training attempt (3-6 hours)

Day 4: Evaluate results, iterate (2-4 hours)

Day 5: Final training, deploy (2-3 hours)

Total: 1-2 weeks of work for one fine-tuned model

ITERATION CYCLE:

Often first fine-tuning isn't perfect. You need to iterate:

Train → Evaluate → If bad, analyze why → Add more data or clean data or adjust hyperparameters → Retrain

Usually takes 2-3 iterations to get good results.

DELIVERY TIPS

This section is practical: students will do this themselves. • OpenAI API example is concrete and modern. • Real timeline and costs help with expectations. • Emphasize that this is not one-and-done: iteration is normal. • The monitoring metrics are important—students should know what to look for.

09. EVALUATION & METRICS

~8 minutes

WHAT'S ON SCREEN

Evaluation framework • Metrics shown: Accuracy, BLEU, Rouge, custom metrics • Before/After comparison • Real evaluation examples

SPEAKING SCRIPT

Here's critical question: How do you know if fine-tuning actually worked?

Can't just guess. You need metrics.

EVALUATION TYPES:

AUTOMATIC METRICS (Fast, objective):

- Accuracy: Percentage of correct predictions
- BLEU/ROUGE: Similarity between generated and expected text
- Task-specific: Precision, recall, F1-score

HUMAN EVALUATION (Slow, subjective):

- Domain experts review outputs
- Rate on quality scale (1-5)
- Check if output is actually useful

Most projects use BOTH.

AUTOMATIC METRICS:

ACCURACY:

Simple: Did the model predict correctly yes/no?

Example (hiring): Did model correctly classify resume as 'Strong match'?
Limitation: Works for classification, not generation

BLEU (Bilingual Evaluation Understudy):

Measures n-gram overlap between predicted and expected text

Range: 0-1 (higher is better)

Use case: Translation, summarization

Limitation: Doesn't capture meaning, just word overlap

ROUGE (Recall-Oriented Understudy for Gisting Evaluation):

Similar to BLEU but emphasizes recall

Use case: Summarization

Limitation: Still just word overlap, misses meaning

Custom metrics:

For your domain, create custom metric

Example (hiring): Metric = 'Did model correctly identify top 3 must-have skills?'

Implement: Parse model output, check if top 3 skills match expert list

HUMAN EVALUATION:

Have domain expert review outputs on scale:

- 1: Completely wrong
- 2: Major errors
- 3: Some errors but useful
- 4: Good, minor issues
- 5: Excellent

Review 50-100 outputs. Calculate average score.

Compare base model average score vs fine-tuned model average score.

EVALUATION PROCESS:

Step 1: Split data

Training: 320 examples (train model)

Validation: 80 examples (evaluate during training, tune hyperparameters)

Test: 50-100 examples (final evaluation, don't touch during training)

Step 2: Train base model for comparison

Keep original base model. We'll compare against it.

Step 3: Train fine-tuned model

Use training + validation set

Step 4: Evaluate on test set

Run BOTH models on test set (100 examples they've never seen)

Calculate metrics for both

Compare

Step 5: Human review

Have expert review ~20-30 examples from each model

Rate quality on 1-5 scale

Calculate average

EVALUATION EXAMPLE: HIRING DOMAIN

Task: Given resume, predict if 'Strong match', 'Weak match', or 'Borderline'

Evaluation metrics:

- Accuracy: What % predictions match expert classification?
- Precision: Of predictions of 'Strong match', how many were actually strong?
- Recall: Of actual strong matches, how many did model find?
- F1: Balance of precision and recall

Test set results:

Base model (GPT-4 zero-shot):

- Accuracy: 72%
- Precision: 0.68
- Recall: 0.75
- Human rating: 3.2/5

Fine-tuned model:

- Accuracy: 85%
- Precision: 0.88
- Recall: 0.82
- Human rating: 4.1/5

Improvement: +13% accuracy, +0.9/5 in human rating

Verdict: Fine-tuning WORKED

WHEN FINE-TUNING FAILS:

Scenario 1: Fine-tuned model WORSE than base

Diagnosis: Usually bad data (wrong labels, poor quality) Fix: Clean data better, remove problematic examples

Scenario 2: Slight improvement, not worth cost

Diagnosis: Domain not distinct enough. Base model already good. Decision: Use prompting instead. Skip fine-tuning.

Scenario 3: Good improvement but strange errors

Diagnosis: Overfitting on certain patterns Fix: Add more diverse examples, use validation set to detect overfitting

COST-BENEFIT IN TERMS OF METRICS:

If fine-tuned is 10-20% better: Worth it (justifies cost)

If fine-tuned is 2-5% better: Maybe not (cost might exceed benefit)

If fine-tuned is worse: Definitely not (fix data, try again)

LONG-TERM EVALUATION:

Don't stop after one evaluation

Monitor performance over time

As new data comes in, occasionally retrain

Compare: Is model still performing well?

DELIVERY TIPS

Metrics are essential for making decisions. Don't skip this. • Real hiring example with specific numbers is powerful. • The 'when fine-tuning fails' section is honest and practical. • Cost-benefit decision framework helps students avoid wasting money. • Long-term monitoring shows this isn't one-time: models need maintenance.

10. REAL-WORLD EXAMPLES

~8 minutes

WHAT'S ON SCREEN

5 Companies using fine-tuning • OpenAI customer examples, Legal tech, Medical AI, Content platforms, Customer service • How each uses it, business impact

SPEAKING SCRIPT

Let me show you how real companies use fine-tuning. This will ground the theory in practice.

COMPANY 1: OPENAI (Internal)

What they do: Fine-tune GPT-4 for different domains

Example domain: Customer service for specific companies

- Collect company's past support tickets + responses from best representatives

- Fine-tune model on this data

Result: Model that sounds like best support rep, follows company policies, uses company terminology

Business impact: Reduces training time for new support staff. Higher customer satisfaction (model has learned best practices). Scales support without hiring.

Cost-benefit: One-time investment of \$5,000-20,000 in fine-tuning. Saves millions in support staff training and improvements.

COMPANY 2: LEGAL TECH (Generic example based on real firms)

What they do: Fine-tune legal-domain model on case law

Data: 50,000 past legal cases with analysis from experienced lawyers

Process: Fine-tune model to learn legal reasoning patterns

Result: Model that reasons like senior lawyer, uses proper legal language, cites precedent

Business impact: Can handle routine legal analysis that would take junior lawyer days. Lets senior lawyers focus on complex cases. Competitive advantage.

Scale: Used 100,000+ times per year → cost of fine-tuning paid back in months

COMPANY 3: MEDICAL AI

What they do: Fine-tune medical-domain model on clinical case descriptions

Data: 100,000 clinical case notes (de-identified) with diagnoses from experienced clinicians

Process: Fine-tune model to learn diagnostic reasoning in this specific hospital

Result: Model that understands this hospital's patient population, diagnostic criteria, terminology

Business impact: Improves diagnostic accuracy by 8-12%. Helps clinicians review cases faster. Reduces diagnostic errors.

Regulatory: Can show fine-tuning data and process to regulators. Increases trust in AI decisions (vs pure LLM, which is seen as 'black box').

COMPANY 4: CONTENT MODERATION PLATFORM

What they do: Fine-tune model to detect specific types of harmful content

Challenge: Base model is general. Doesn't understand nuances of specific platform's rules.

Example: Tweet might seem insulting but is actually joke between friends. Base model flags it. Fine-tuned model understands context.

Data: 50,000 past moderation decisions from human moderators

Process: Fine-tune model on 'should this be removed' decisions
Result: Model matches human moderator decisions 92% of time

Business impact: Catches 95% of truly harmful content while reducing false positives (things that seem bad but aren't).
Improves user experience (fewer good posts removed).

COMPANY 5: RECRUITMENT PLATFORM

What they do: Fine-tune model to understand recruiting domain

Example: Given resume, generate personalized job recommendations

Data: 100,000 past candidate-to-job matches (what candidates applied for, what were successful placements)

Process: Fine-tune model on these matches

Result: Model understands: which skills transfer between roles, what makes a good match, industry terminology

Business impact: Better job recommendations (more candidates placed successfully). Candidates spend less time searching. Companies find right candidates faster. Network effect (more matches → more value).

PATTERNS IN ALL FIVE:

- 1) They all had domain-specific data (100+ to 100,000 examples)
- 2) Domain was distinct enough to benefit from specialization
- 3) Model was used frequently enough to justify investment
- 4) Evaluation showed clear improvement over base model
- 5) ROI was positive within months

WHAT MADE THEM SUCCESSFUL:

- ✓ Clear problem (support inefficiency, legal complexity, diagnostic errors, moderation workload, recruitment challenges)
- ✓ Good data (real examples from domain experts)
- ✓ Rigorous evaluation (metrics showed improvement)
- ✓ Clear business case (cost savings or new capability)
- ✓ Long-term commitment (not one-time project, ongoing improvement)

WHAT FAILED PROJECTS DO WRONG:

✗ No clear problem (trying to fine-tune 'just because')
✗ Bad data (insufficient, incorrect, or biased examples)
✗ No evaluation (blindly assume it's better)
✗ Unclear ROI (spend \$10k, save \$1k)
✗ Short-term thinking (train once, never update)

DELIVERY TIPS

Five real companies across different industries shows this is mainstream. • Each example shows: problem → solution → business impact. Students should see pattern. • Be honest about failures too. • Emphasize: success = good data + clear problem + rigorous evaluation + business case.

11. RAG + FINE-TUNING COMBINED

~5 minutes

WHAT'S ON SCREEN

Combined approach diagram • Fine-tuned model (learned domain) + RAG (current documents) • Synergy shown • Real-world powerful combination

SPEAKING SCRIPT

Now here's where the magic happens: combining fine-tuning AND RAG.

FINE-TUNING ALONE:

Pro: Fast, specialized reasoning

Con: Not fresh, knowledge locked in training data

RAG ALONE:

Pro: Always current, can cite sources

Con: Needs time to retrieve, no domain specialization

COMBINED:

Pro: Fast specialized reasoning PLUS fresh current information

Con: More complex, requires both systems

HOW IT WORKS:

Step 1: Fine-tune model on domain examples

Result: Model understands your domain deeply

Step 2: At query time, ALSO retrieve relevant documents

Result: Model has current context

Step 3: Fine-tuned model reads question + retrieved documents

Result: Model reasons using both learned patterns AND current data

REAL EXAMPLE: MEDICAL AI (The ultimate version)

Step 1: Fine-tune on 100,000 clinical cases

→ Model learns diagnostic reasoning, medical terminology, what questions to ask

Step 2: At query time, retrieve latest research on patient's symptoms from PubMed

→ Model has current evidence

Step 3: Model analyzes symptom + latest research using learned diagnostic patterns

→ Model recommends diagnosis grounded in current evidence AND specialized reasoning

SYNERGY:

Without fine-tuning (RAG only):
Retrieves research papers
Summarizes them (generically)
Might miss important nuances
Could recommend based on recent paper even if contradicts established practice

Without RAG (fine-tuning only):
Reasons using learned patterns
But doesn't know about last week's study
Might recommend outdated treatment

With both:
Reasons using established patterns (fine-tuning)
But aware of recent evidence (RAG)
Recommends combining best of both

ANOTHER EXAMPLE: LEGAL AI (Production system)

Fine-tuning: Model trained on 50,000 past cases + legal analysis

→ Understands legal reasoning, precedent, how to structure arguments

RAG: At query time, retrieve most recent 5 similar cases from database

→ Model has current precedent

Result: Model generates legal analysis informed by:

- Deep understanding of legal reasoning (fine-tuning)■r/>- Most recent relevant cases (RAG)
- With citations so lawyer can verify (RAG)■r/>- Written like senior lawyer (fine-tuning)

COST ANALYSIS:

Fine-tuning investment: \$5,000 - \$20,000 (one-time)

RAG infrastructure: \$50-200/month (ongoing)

Total: ~\$5,000-30,000 first year, ~\$600-2,400/year ongoing

Compare to hiring expert lawyer: \$150,000-300,000/year

ROI: Pays for itself in months.

WHEN TO USE COMBINED APPROACH:

Use when:

- Domain is specialized (legal, medical)
- Knowledge is both stable AND changing (well-established practices + new research)
- Speed matters (fine-tuning) but freshness also matters (RAG)
- You have budget for both

Examples: Legal, medical, financial, regulatory domains

Don't combine when:

- One alone is sufficient■r/>- Budget is tight (choose one)■r/>- Complexity isn't worth the benefit

DELIVERY TIPS

This is where Sessions 5 and 6 converge. Powerful teaching moment. • Medical and legal examples show this isn't theoretical—real companies do this. • The synergy explanation is crucial: explain why both together is better than either alone. • Cost analysis shows ROI. • Students should understand: this is the frontier of production AI.

12. LAB EXERCISE OVERVIEW

~3 minutes

WHAT'S ON SCREEN

Lab title: 'Create Your Fine-Tuning Training Data' • Steps: Collect examples, Format JSONL, Evaluate quality, Simulate training

SPEAKING SCRIPT

Alright, lab time. You're going to understand fine-tuning by preparing training data and simulating a fine-tuning process.

YOUR TASK: Create a fine-tuning dataset for your hiring domain. Prepare training data. Understand the format. Evaluate quality. Discuss how to fine-tune and what results might look like.

STEP 1: CREATE TRAINING EXAMPLES (3 min)

Generate or collect 20-30 prompt-completion pairs for your domain

Example: (Resume description → Hiring recommendation)

STEP 2: FORMAT AS JSONL (2 min)

Convert to JSONL format that fine-tuning APIs expect

STEP 3: QUALITY CHECK (2 min)

Review your data for consistency, correctness, and diversity

STEP 4: SPLIT TRAIN/VALIDATION (1 min)

80% training, 20% validation

STEP 5: ESTIMATE COSTS & RESULTS (2 min)

Calculate: How much would this cost to fine-tune? What results would you expect?

STEP 6: COMPARE: FINE-TUNING vs RAG vs PROMPTING (2 min)

For your use case, which approach is best? Why?

DELIVERABLES:

1. 20-30 training examples (your domain) 2. JSONL formatted data 3. Data quality assessment 4. Train/validation split 5. Cost and timeline estimate 6. Comparison: fine-tuning vs RAG vs prompting for your use case

DELIVERY TIPS

Step-by-step makes it manageable. • This lab is about understanding fine-tuning data requirements, not actually training (too expensive/complex for students). • Emphasize data quality: garbage in, garbage out. • Cost estimation is practical. • Comparison exercise makes them think critically about when to use which technique.

13. KEY TAKEAWAYS FROM SESSION 6

~5 minutes

WHAT'S ON SCREEN

6 Core Principles • Fine-tuning definition, when to use, data is critical, evaluation matters, combined with RAG, real ROI

SPEAKING SCRIPT

Takeaway One: Fine-Tuning Teaches Models Your Domain

Not by changing prompts. By showing examples and letting model learn patterns. Different from prompting. Permanent change to model behavior.

Takeaway Two: Fine-Tuning Is NOT the Default

Sometimes prompting is enough. Sometimes RAG is better. Evaluate the decision: Do you have data? Is domain distinct? Is speed critical? Is knowledge stable?

Takeaway Three: Data Is 80% of the Work

Training is fast and automated. But data preparation is hard. Good data → good model. Bad data → bad model, no amount of tuning fixes it.

Takeaway Four: Evaluation Determines Success

Can't guess if it worked. You need metrics. Automatic and human. Compare to baseline. Is improvement meaningful? Is ROI positive?

Takeaway Five: Combine RAG + Fine-Tuning for Maximum Power

Fine-tuning: Learned specialization. RAG: Current information. Together: Fast, specialized, fresh, verifiable.

Takeaway Six: This Is Production AI

Real companies do exactly this. Legal, medical, customer service, content moderation. You're learning what billion-dollar companies build.

DELIVERY TIPS

These six principles tie Sessions 1-6 together. • Emphasis on data being 80% of the work is important—resets expectations. • RAG + fine-tuning as 'maximum power' connects to Session 5. • Real companies doing this validates importance.

14. COMING UP NEXT

~2 minutes

WHAT'S ON SCREEN

Dark blue background • Title: 'Session 7: Evaluation & Benchmarking' • Preview bullets • Closing remarks

SPEAKING SCRIPT

Session 7 is where you systematically measure AI system performance. Not just 'is it good?' but 'exactly how good, compared to what, and what metrics matter?'

By now you've built: specialized prompts (Session 3), structured pipelines (Session 4), grounded systems (Session 5), and fine-tuned models (Session 6).

Session 7: How do you prove these are better? How do you measure? What's the benchmark?

You'll learn: Creating evaluation benchmarks, running A/B tests, understanding when improvements are statistically significant vs lucky, and building evaluation frameworks you can use repeatedly.

In the lab, you'll evaluate your full system (prompt + structure + retrieval + fine-tuning) against benchmarks and competing approaches.

Thank you all for today. You've learned to build specialized AI systems. See you in Session 7.

DELIVERY TIPS

Session 7 natural progression: You've built systems, now measure them. • Evaluation framework for building better systems. • Lab brings everything together for systematic comparison.